Scanner Class

Taking user input in Java

Scanner Class

Scanner is used to read input from the user.

It belongs to the java.util package.

It can read:

- String
- int
- double
- float
- char
- boolean

Importing Scanner

import java.util.Scanner;

Imports the Scanner class.

Required before using it.

Scanner is non-static, we must create an object.

Creating Scanner Object

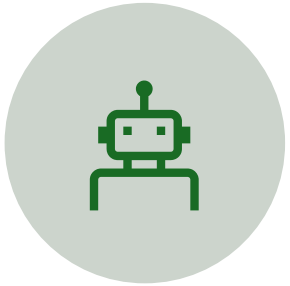
- Scanner sc = new Scanner(System.in);
- **Scanner** → class name
- **sc** → object reference
- **new** → creates object in heap
- **System.in** → keyboard input

System.in

Predefined static InputStream object.

Represents **standard input (keyboard)**.

Tells Scanner where to read data from.



Without System.in, Scanner won’t know the input source.

Reading Different Data Types

```
String name = sc.nextLine(); // String
```

```
int age = sc.nextInt(); // Integer
```

```
double salary = sc.nextDouble(); // Double
```

```
float marks = sc.nextFloat(); // Float
```

```
boolean flag = sc.nextBoolean(); // Boolean
```

Example

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);

        sc.close();
    }
}
```

Output

```
Enter your name: Bilal
Enter your age: 29
Name: Bilal
Age: 29

=== Code Execution Successful ===
```

Other Input Sources

Source	**Example**
Keyboard	new Scanner(System.in)
File	new Scanner(new File("data.txt"))
String	new Scanner("10 20 30")
Network	new Scanner(socket.getInputStream())

Practice Tasks

Take two integers and print their sum.

Take a name and city, print a welcome message.

Take marks for 3 subjects and calculate average.

Take a number and print whether it is even or odd.

Summary

Scanner is used to take input in Java.

It reads different data types.

Requires object creation.

Uses System.in for keyboard input.

Makes programs interactive.

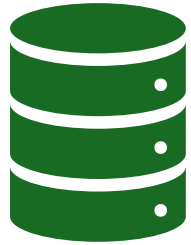
Type Casting in Java

Converting one data type into another data type

Used cases

Performing calculations

Assigning incompatible types



Controlling precision / memory

Types of Type Casting in Java

Implicit Casting (Automatic)

Explicit Casting (Manual)

Implicit Casting (Widening)

- Smaller → Larger data type
- Done automatically by Java.
- byte → short → int → long → float → double
- **Output:** 25.0, decimal point added

```
class test {
    public static void main(String[] args) {
        int x = 25;
        double y = x;   // Automatic casting

        System.out.println(y);
    }
}
```

Why Called Widening?

- Memory size increases
- No data loss occurs

From	To	Safe?
int	double	✓
char	int	✓

Explicit Casting (Widening) > Usually

- Larger → Smaller data type
- Must be done manually.
- **Output:** 9, Decimal part is removed.

```
class test {
    public static void main(String[] args) {
        double x = 9.7;
        int y = (int) x;   // Manual casting

        System.out.println(y);
    }
}
```

Data Loss in Narrowing

- When casting large → small:
- Fraction lost
- Overflow possible
- **Output:** -124, negative sign (overflow)

```
class test {
    public static void main(String[] args) {
        int x = 132;
        byte y = (byte) x; // Manual casting
        System.out.println(y);
    }
}
```

Implicit vs Explicit

Feature	**Implicit**	**Explicit**
Done by	Java	Programmer
Direction	Small → Large	Large → Small
Data Loss	No	Possible
Syntax	Automatic	Manual cast

Example

```
class test {
   public static void main(String[] args) {

      int marks = 85;
      double percentage = marks;

      double price = 99.99;
      int rounded = (int) price;

      System.out.println(percentage);
      System.out.println(rounded);
   }
}
```

Types of Operators in Java

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Unary Operators

Arithmetic Operators

Operator	**Meaning**
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example

```
class test {
    public static void main(String[] args) {
        int a = 10, b = 3;

        System.out.println(a + b);
        System.out.println(a - b);
        System.out.println(a * b);
        System.out.println(a / b);
        System.out.println(a % b);
    }
}
```

Relational Operators

Operator	Meaning
==	Equal to
!=	Not equal
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

Example

```
class test {
    public static void main(String[] args) {
        int a = 10, b = 5;

        System.out.println(a > b);
        System.out.println(a == b);
        System.out.println(a != b);
    }
}
```

Logical Operators

Operator	**Meaning**
&&	AND
||	OR
!	NOT

Example

```
class test {
    public static void main(String[] args) {
        int age = 20;

        System.out.println(age > 18 && age < 30);
        System.out.println(age < 18 || age > 60);
        System.out.println(!(age > 18));
    }
}
```

Assignment Operators

Operator	Example
=	x = 5
+=	x += 3
-=	x -= 2
*=	x *= 4
/=	x /= 2

Example

```
class test {
    public static void main(String[] args) {
        int x = 10;

        x += 5;
        System.out.println(x);

        x *= 2;
        System.out.println(x);
    }
}
```

Unary Operators

Operator	Meaning
++	Increment
--	Decrement
+	Unary plus
-	Unary minus

Example

```
class test {
    public static void main(String[] args) {
        int x = 5;

        System.out.println(x++);
        System.out.println(++x);
    }
}
```

Control Structures in Java

Control structures determine the flow of program execution.

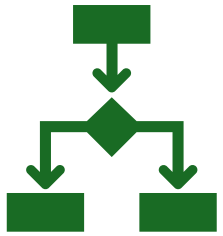
Control Structures Decides

- Which statement runs
- How many times it runs
- Under what condition it runs

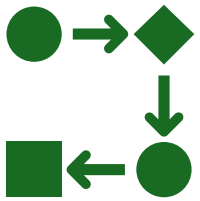
Why Control Structures Are Needed

Without control structures:	With control structures we can:
• Programs run sequentially only • No decision making • No repetition	• Make choices • Repeat tasks • Build logic

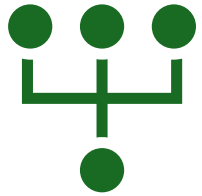
Types of Control Structures



Decision Making (Selection)



Looping (Iteration)



Branching (Jump Statements)

Decision-Making Structures

If statement

Syntax

```java
if(condition) {
    // code
}
```

Example

```java
int age = 20;

if(age >= 18) {
    System.out.println("Eligible to vote");
}
```

if-else Statement

Syntax

java

```java
if(condition) {
    // true block
} else {
    // false block
}
```

Example

java

```java
int num = 5;

if(num % 2 == 0)
    System.out.println("Even");
else
    System.out.println("Odd");
```

else-if Ladder

```
int marks = 75;


if(marks >= 90)
    System.out.println("A");
else if(marks >= 70)
    System.out.println("B");
else
    System.out.println("C");
```

Nested if

```
int age = 20;
int id = 1;

if(age >= 18) {
    if(id == 1)
        System.out.println("Allowed");
}
```

switch Statement

Switch

Syntax

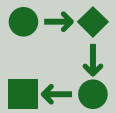
java

```java
switch(value) {
    case 1:
        // code
        break;
    case 2:
        // code
        break;
    default:
        // code
}
```

```java
int day = 2;

switch(day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Invalid");
}
```

Loops in Java



for → known number of iterations.

while → unknown number of iterations, checks condition first.

do-while → executes at least once, then checks condition.

For Loop Example

For Loop

```java
for(int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

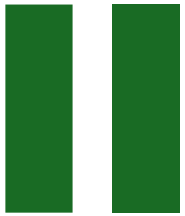
While Loop

```java
int i = 1;
while(i <= 5) {
    System.out.println(i);
    i++;
}
```

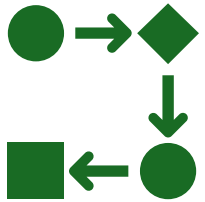
Do While Loop

```java
int i = 1;
do {
    System.out.println(i);
    i++;
} while(i <= 5);
```

Branching Statements



break → exit loop immediately.



continue → skip current iteration.

return → exit from a method.

Example

```
public class test {
   public static void main(String[] args){

      System.out.println("Break Output");
      for (int i = 0; i < 5; i ++){
         if (i==3)
         break;
         System.out.println(i);
      }
      System.out.println("Continue Output");
      for (int i = 0; i < 5; i ++){
         if (i==3)
         continue;
         System.out.println(i);
      }
      System.out.println("Return Output");
      for (int i = 0; i < 5; i ++){
         if (i==3)
         return;
         System.out.println(i);
      }
}}
```

Output

```
Break Output
0
1
2
Continue Output
0
1
2
4
Return Output
0
1
2

=== Code Execution Successful ===
```